

Deep Learning Report

Lyheng LAO, Guang YANG

May 11, 2026

Abstract

Recurrent Neural Networks (RNNs) with an encoder-decoder architecture have shown promising results in the field of machine translation. The article "*Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*" proposes a novel neural network architecture for statistical machine translation, termed the RNN Encoder-Decoder [CvMG⁺14]. At the same time, it also gave rise to the GRU model. The primary aim of them is to enhance the conditional probability of generating a target sequence given a source sequence. This is achieved by jointly training the encoder and decoder to optimize this conditional probability. This report aims to analyze the article by conducting some simple experiments to better understand the proposed method and to determine if the *RNN encoder-decoder* achieves better translation accuracy than *Statistical Machine Translation (SMT)*.

Contents

Contents	1
1 Introduction	2
2 RNN	2
2.1 Forward Propagation	3
2.2 Loss Function and One-Hot Code	3
2.3 Backpropagation and Update Formulas	4
3 RNN Encoder-Decoder	5
3.1 GRU	6
3.2 LSTM	7
3.3 Attention	8
3.4 BLUE score	10
4 Some Experiment on the Proposed Structure	10
4.1 GRU vs LSTM: Why Introduce new Structure rather than using LSTM	10
4.2 Attention	11
4.3 STM	13
5 Conclusion	14
References	14
A BLUE Score	15

1 Introduction

Recurrent Neural Networks (RNNs) have become a fundamental tool in the field of artificial intelligence for handling sequential data. Among the various architectures developed to leverage the capabilities of RNNs, the encoder-decoder model stands out as a particularly powerful framework. Initially proposed for machine translation, this architecture has significantly advanced the performance of numerous sequence-to-sequence tasks, making it a cornerstone in natural language processing and other domains requiring the transformation of sequences.

The encoder-decoder architecture was first introduced by Ilya Sutskever, Oriol Vinyals, and Quoc V. Le in their seminal 2014 paper, "Sequence to Sequence Learning with Neural Networks" [SVL14]. This architecture effectively addresses the limitations of standard RNNs in dealing with variable-length input and output sequences by employing two distinct RNNs: the encoder and the decoder. The encoder processes the input sequence and transforms it into a fixed-size context vector, which encapsulates the semantic information of the input. The decoder then uses this context vector to generate the target sequence, one element at a time.

Despite its initial success, the basic encoder-decoder model faced challenges when handling long sequences, as the fixed-size context vector could lose critical information. This limitation led to the introduction of attention mechanisms by Bahdanau et al. in 2014 [BCB14]. Attention mechanisms allow the decoder to access different parts of the input sequence dynamically, thereby improving the model's ability to handle longer and more complex sequences. This innovation has not only enhanced machine translation systems but also broadened the applicability of encoder-decoder models to tasks such as text summarization, image captioning, and speech recognition.

An important enhancement to the RNN architecture was the development of Long Short-Term Memory (LSTM) networks by Sepp Hochreiter and Jürgen Schmidhuber in 1997 [HS97]. LSTMs introduced memory cells that can maintain information over long periods and gating mechanisms that control the flow of information into and out of these cells, effectively mitigating the vanishing gradient problem. In 2014, Kyunghyun Cho and colleagues proposed the Gated Recurrent Unit (GRU), a simplified version of the LSTM that uses fewer gates while retaining similar performance [CvMG⁺14]. Both LSTMs and GRUs have become essential components of encoder-decoder models, providing robust solutions for sequence modeling tasks.

To evaluate the performance of these models, metrics such as the BLEU (Bilingual Evaluation Understudy) score are commonly used. The BLEU score, introduced by Papineni et al. in 2002, measures the accuracy of machine-generated translations compared to reference translations, considering factors like precision and brevity [PRWZ02]. This metric has become a standard in assessing the quality of machine translation systems and other sequence-to-sequence models.

In this report, we aim to analyze the RNN encoder-decoder architecture and its enhancements, specifically focusing on the GRU and LSTM models. We will conduct a series of experiments to evaluate their performance in machine translation tasks, comparing their efficiency, accuracy, and computational requirements. Additionally, we will explore the impact of the attention mechanism on these models, particularly how it enhances their ability to handle longer sequences by dynamically focusing on relevant parts of the input.

2 RNN

This section, we will provide a detailed introduction to the RNN model, which lays the foundation for building the RNN encoder-decoder.

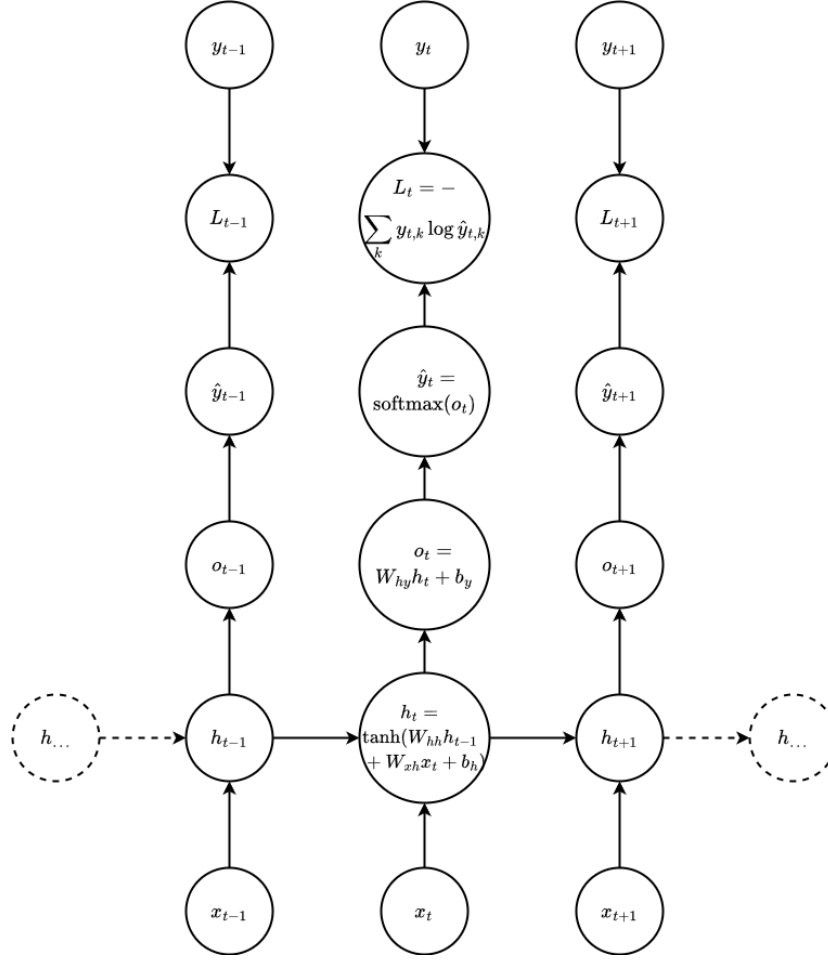


Figure 1: RNN

2.1 Forward Propagation

Assume our RNN receives an input x_t at each time step, updates its hidden state h_t , and finally produces an output y_t . For a single-layer RNN, these updates can be expressed as:

- **Hidden state update:**

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

where W_{hh} is the weight matrix from hidden state to hidden state, W_{xh} is the weight matrix from input to hidden state, and b_h is the bias of the hidden layer.

- **Output layer:**

$$\hat{y}_t = \text{softmax}(o_t), \text{ and } o_t = W_{hy}h_t + b_y,$$

where W_{hy} is the weight matrix from the hidden state to the output layer, and b_y is the bias of the output layer.

2.2 Loss Function and One-Hot Code

In RNNs, there are various options for choosing the loss function. Here, we will only discuss *cross-entropy*. The discussion for choosing other loss functions is similar.

For classification tasks, the cross-entropy loss at each time step is calculated as:

$$L_t = - \sum_k y_{t,k} \log \hat{y}_{t,k}$$

where $y_{t,k}$ is the *one-hot encoded* target output at time step t , and $\hat{y}_{t,k}$ is the predicted probability by the model.

In fact, y_t is a one-hot encoded vector, where only the position of the true category is 1, and the rest are 0. For example, if there are five possible categories and the true category is the third, then y_t might look like this:

$$y_t = [0, 0, 1, 0, 0].$$

Moreover, $\hat{y}_t$ represents the model's predicted probabilities for each category. For example, the model's output for the above five categories might be:

$$\hat{y}_t = [0.1, 0.2, 0.4, 0.2, 0.1].$$

Index k here refers to the position in these vectors, identifying a specific category. For the example above, k could take values from 1 to 5, corresponding to the five categories.

If for some time step t the true category k , the model's predicted probability $\hat{y}_{t,k}$ is close to 1, then $y_{t,k} \log \hat{y}_{t,k}$ contributes very little to the loss (because $\log(1) = 0$). If $\hat{y}_{t,k}$ is close to 0, the loss becomes very large, as $\log(0)$ tends toward negative infinity, thereby penalizing such inaccurate predictions.

By minimizing this type of loss, the model learns how to correctly predict the probability of each category, making the predicted probability distribution as close as possible to the true probability distribution.

2.3 Backpropagation and Update Formulas

Using the corresponding gradients, ones can update the weights and biases of the network. If using a learning rate η for standard gradient descent updates, the update formulas are:

$$\begin{aligned} W_{hy} &\leftarrow W_{hy} - \eta \frac{\partial L}{\partial W_{hy}}, & \frac{\partial L}{\partial W_{hy}} &= \sum_t \frac{\partial L}{\partial o_t} \frac{\partial o_t}{\partial W_{hy}} = \sum_t (\hat{y}_t - y_t) h_t^T; \\ b_y &\leftarrow b_y - \eta \frac{\partial L}{\partial b_y}, & \frac{\partial L}{\partial b_y} &= \sum_t (\hat{y}_t - y_t); \\ W_{hh} &\leftarrow W_{hh} - \eta \frac{\partial L}{\partial W_{hh}}, & \frac{\partial L}{\partial W_{hh}} &= \sum_t \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}} = \sum_t \frac{\partial L}{\partial h_t} \cdot \tanh'(z_t) \cdot h_{t-1}^T; \\ W_{xh} &\leftarrow W_{xh} - \eta \frac{\partial L}{\partial W_{xh}}, & \frac{\partial L}{\partial W_{xh}} &= \sum_t \frac{\partial L}{\partial h_t} \cdot \tanh'(z_t) \cdot x_t^T; \\ b_h &\leftarrow b_h - \eta \frac{\partial L}{\partial b_h}, & \frac{\partial L}{\partial b_h} &= \sum_t \frac{\partial L}{\partial h_t} \cdot \tanh'(z_t), \\ \frac{\partial L}{\partial h_t} &= W_{hy}^T (\hat{y}_t - y_t) + \left(W_{hh}^T \left(\frac{\partial L}{\partial h_{t+1}} \odot \tanh'(h_{t+1}) \right) \right). \end{aligned}$$

where $z_t = W_{hh} h_{t-1} + W_{xh} x_t + b_h$ is the input before applying the activation function, and $\odot$ represents the Hadamard product (element-wise multiplication).

3 RNN Encoder–Decoder

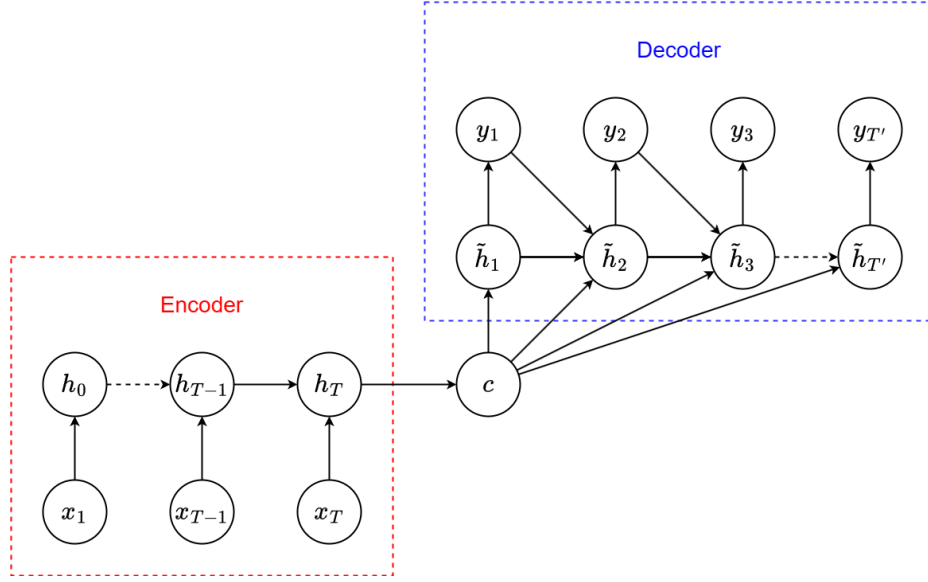


Figure 2: RNN Encoder–Decoder

The *RNN Encoder-Decoder* is an architecture based on RNNs, widely used in sequence-to-sequence tasks such as machine translation, text generation, and speech recognition. This architecture introduces two parts, the *Encoder* and the *Decoder*, to overcome some of the limitations of traditional RNNs when handling long sequences:

- **Encoder:** The encoder is an RNN that processes each element of the input sequence step by step and updates its hidden state. The last hidden state is regarded as the representation of the input sequence. It is responsible for reading the input sequence and encoding it into a fixed-length context vector. Its hidden state compute by :

$$h_t = f(h_{t-1}, x_t). \quad (1)$$

- **Decoder:** The decoder is also an RNN that generates each element of the target sequence step by step, relying on the previously generated elements and the current hidden state. It is responsible for reading the input sequence and encoding it into a fixed-length context vector. Compared by the encoder, its hidden state is using the context vector $\mathbf{c}$ generated by the encoder:

$$\tilde{h}_t = f(\tilde{h}_{t-1}, y_{t-1}, \mathbf{c}). \quad (2)$$

Compared to traditional RNNs, RNN Encoder–Decoder has significant advantages.

For sequence-to-Sequence processing capability, RNNs are typically used for tasks with fixed-length input and output sequences, such as time series prediction. However, RNN Encoder-Decoder is capable of handling variable-length input and output sequences, making it an ideal choice for sequence-to-sequence tasks. The encoder encodes the variable-length input sequence into a fixed-length vector, and the decoder generates a variable-length output sequence from this vector.

Moreover, the traditional RNNs process the entire input sequence directly and cannot effectively capture long-distance dependencies, while RNN Encoder-Decoder passes the global

information of the input sequence through the context vector, allowing the model to better capture long-distance dependencies when generating the output sequence.

Next, we will discuss several related models of the RNN Encoder-Decoder.

3.1 GRU

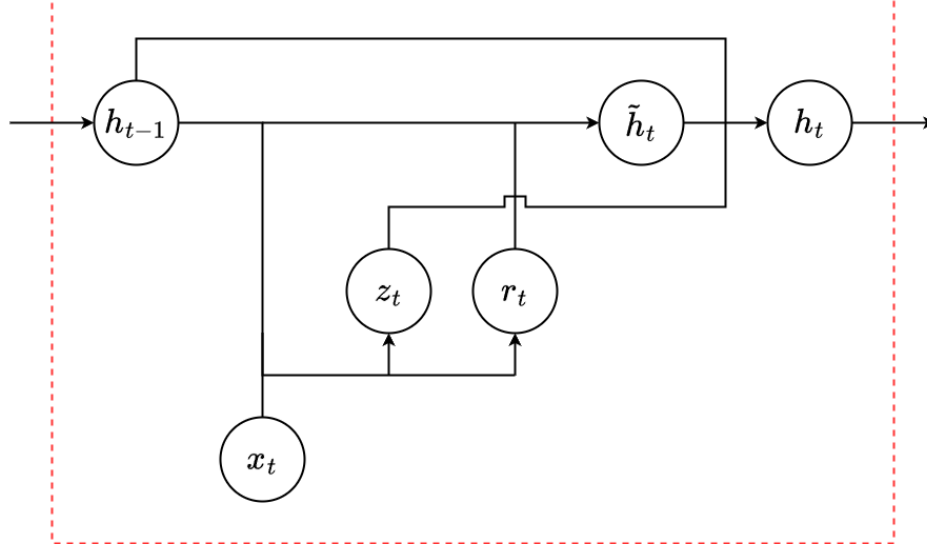


Figure 3: GRU

GRU is a variant of the RNN. The main purpose to introduce it in RNN encoder decoder is to propose a new machine translation. In general, GRU introduces gating mechanisms to control the flow of information, thereby better capturing long-distance dependencies.

The standard GRU is composed of the following main parts:

- **Update Gate (z_t):** Determines how the previous state and the current candidate state are combined to update the current state.
- **Reset Gate (r_t):** Determines how the previous state influences the candidate state.

At each time step t , the state update process of the GRU is as follows:

(a) **Calculate the update gate z_t :**

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

where σ is the sigmoid function, W_z is the weight matrix, h_{t-1} is the previous hidden state, and x_t is the current input.

(b) **Calculate the reset gate r_t :**

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

Similarly, W_r is the weight matrix.

(c) **Calculate the candidate hidden state $\tilde{h}_t$:**

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \quad (3)$$

where W is the weight matrix, $*$ denotes element-wise multiplication, and $\tanh$ is the hyperbolic tangent function.

(d) **Calculate the new hidden state h_t :**

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t, \quad (4)$$

Here, h_t is the hidden state at the current time step.

Now take a look about the different of *LSTM* and proposed model

3.2 LSTM

LSTM (Long Short-Term Memory) is a special type of RNN, designed to address the issues of vanishing and exploding gradients encountered by traditional RNNs when processing long sequences. LSTM, introduced by Hochreiter and Schmidhuber in 1997, has since undergone numerous improvements and extensions [HS97].

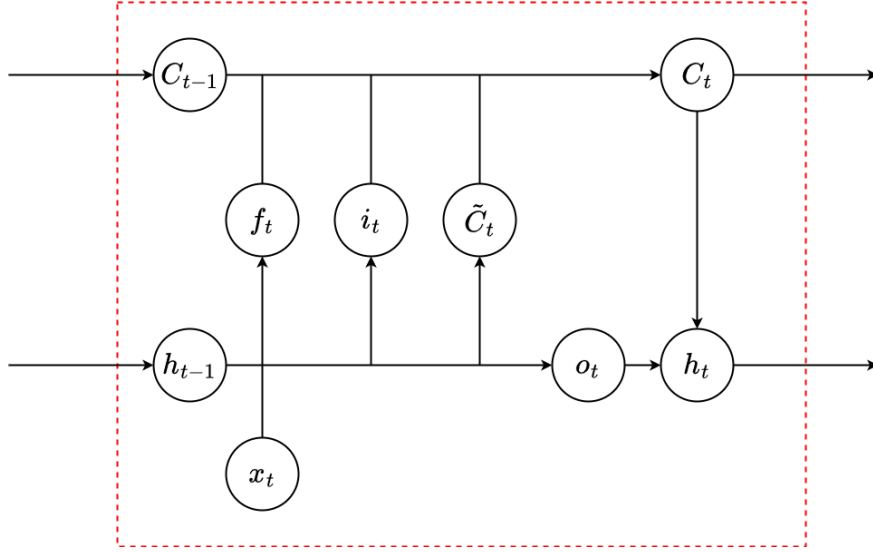


Figure 4: LSTM

The following are the main characteristics and working principles of LSTM:

- **Cell State:** The core of LSTM is a structure called the cell state, which can transmit information between time steps. The cell state can be seen as an "information pipeline," where information can be retained, updated, or discarded through a series of gates.
- **Gating Mechanism:** LSTM controls the flow of information through three gates (**input gate**, **forget gate**, and **output gate**).

Here are details about the corresponding working principles:

Forget Gate: Decides which information in the cell state needs to be forgotten at the current time step. The output of the forget gate is a value between 0 and 1, multiplied by the cell state's state to determine the amount of information to be forgotten.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f).$$

Input Gate: Decides how much new information needs to be added to the cell state. The input gate has two parts: one determines the information to be updated, and the other determines the amount of information to be updated.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i),$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C).$$

Update Cell State: Combines the information from the forget gate and the input gate to update the cell state. Use $*$ to represent the Hadamard product (element-wise multiplication), then

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t.$$

Output Gate: Decides the information to be output and the hidden state to be passed to the next time step.

$$\begin{aligned} o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o), \\ h_t &= o_t * \tanh(C_t). \end{aligned}$$

Remark 3.1. (a) GRU has one fewer gate than LSTM (GRU has two gates: the reset gate and the update gate, while LSTM has three gates: the input gate, the forget gate, and the output gate). This indeed simplifies the structural complexity.

(b) Due to the reduced number of gates, GRU is simpler in terms of computation and implementation, resulting in relatively shorter processing time. This makes GRU more advantageous than LSTM in certain applications.

Remark 3.2. (a) Consider the equation (3) in GRU; when the reset gate r_t is close to 0, the candidate hidden state $\tilde{h}_t$ of the GRU mainly relies on the current input x_t , almost ignoring the previous hidden state h_{t-1} . *This is similar to the role of the forget gate in LSTM, both aiming to control the degree of dependency on past information.*

(b) Consider the equation (4) in GRU; the update gate z_t determines whether the current hidden state depends more on the previous hidden state or the new candidate hidden state. *This mechanism achieves selective retention and updating of information to some extent, similar to the combined functions of the input gate and output gate in LSTM.*

3.3 Attention

The attention mechanism in RNN is an improved method designed to address the long dependency problem encountered by RNNs when processing long sequences [BCB14].

In the case of long sequences, traditional RNNs may forget early information due to long dependencies. The attention mechanism calculates the importance weights of each time step in the input sequence, allowing the model to "attend" to different input time steps when generating each output. This method improves the model's performance and accuracy. It compose of 3 main part:

- **Query (Q):** The query is the representation of the current step in the sequence processing. For instance, in machine translation, this could be the representation of the current word being translated.
- **Key (K):** The key is a representation of all the input elements. It provides the context against which the query is compared to find relevant information.
- **Value (V):** The value is the information that corresponds to each key. It represents the actual data that will be attended to.

where those values are used to calculate the *score function* of Attention mechanism.

The following is a detailed explanation of the attention mechanism and its related formulas. Suppose the input sequence is $(x_1, x_2, \dots, x_T)$, the hidden states of the encoder and decoder are $(h_1, h_2, \dots, h_T)$ and $(s_1, s_2, \dots, s_{T'})$, and the output sequence is $(y_1, y_2, \dots, y_{T'})$, where T' is the length of the output sequence.

For each output time step i , calculate the attention weight $a_{i,j}$ for each time step j in the input sequence. Write

$$e_{i,j} = \text{score}(s_{i-1}, h_j),$$

where $\text{score}(s_{i-1}, h_j)$ is a scoring function that calculates the similarity between the hidden states s_{i-1} and h_j . Then, use the softmax function to normalize the scores into attention weights:

$$a_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^T \exp(e_{i,k})}.$$

Using weights $a_{i,j}$ and hidden state of the encoder h_j , we obtain the context vector c_i and s_i

$$c_i = \sum_{j=1}^T a_{ij} h_j, \quad s_i = f(s_{i-1}, y_{i-1}, c_i).$$

Remark 3.3. In the Attention mechanism, the query Q corresponds to the decoder's hidden state at the current time step s_{i-1} . The key K and value V corresponds to all the hidden states of the encoder $(h_1, h_2, \dots, h_T)$.

Remark 3.4. We will introduce the relevant practices in the next section, where we use the Bahdanau-Attention: for single $q = s_{i-1}$ and a set of keys $k = (h_1, \dots, h_n)$,

$$\text{scores} : e_{i,*} = V^T (\tanh(W(q) + U(k) + b_a)) = V^T \tanh(W s_{i-1} + U h_* + b_a), \text{ for all keys,}$$

where W is the weight matrix for the decoder's hidden state, U is the weight matrix for the encoder's hidden state, V is the weight vector, and b_a is the bias vector.

Remark 3.5. Traditional RNNs can only utilize information from the beginning of the sequence up to the current step, but they cannot use future (subsequent) information. As an improvement, Figure 5(a) applies BI-RNN (Bidirectional Recurrent Neural Network) [GS05]. A BI-RNN employs two independent RNNs on the sequence data: one processes the sequence from front to back (forward RNN), and the other processes it from back to front (backward RNN). The outputs of both RNNs are combined, allowing each time step to consider both past and future information simultaneously.

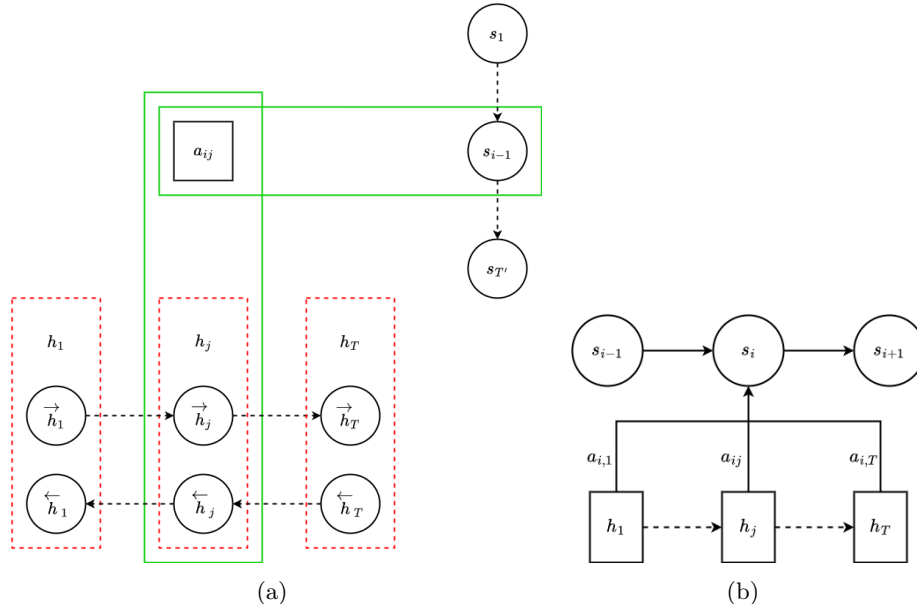


Figure 5: Attention

3.4 BLUE score

The BLEU (Bilingual Evaluation Understudy) score is an automatic metric used to evaluate the quality of texts translated by machine translation systems. Introduced in 2002 by Kishore Papineni and colleagues at IBM, it is one of the most commonly used measures for assessing the performance of machine translation systems [PRWZ02]. Let us have a quick talk about how it work:

- **N-gram Matching:** The BLEU score is based on matching n-grams (sequences of n words) between the machine translation and one or more reference translations. N-grams can be unigrams (individual words), bigrams (pairs of words), trigrams, etc.
- **N-gram Precision:** BLEU calculates the precision of n-grams, which is the number of n-grams in the machine translation that also appear in the reference translation, divided by the total number of n-grams in the machine translation.
- **Brevity Penalty:** To avoid overly short translations, a brevity penalty is applied if the length of the machine translation is shorter than the reference.

The BLEU score is calculated by combining the precision of n-grams for different n (usually up to 4-grams) with a brevity penalty. The general formula is:

$$BLEU = BP * \exp\left(\sum_{n=1}^N w_n \log(p_n)\right),$$

where:

- BP is Brevity Penalty,
- p_n is the modified precision of n-grams,
- w_n is the weight assigned to each n-gram (often equal).

For the specific calculation formulas in this subsection, please refer to the appendix.

4 Some Experiment on the Proposed Structure

This section involves code practice; if you need the corresponding code, please click [here](#).

4.1 GRU vs LSTM: Why Introduce new Structure rather than using LSTM

As mentioned previously, the structure of the proposed model GRU is simpler than LSTM. Now, let's compare these two architectures through some simple implementations on a translation task to evaluate their efficiency and determine which one is more favorable.

GRU and LSTM scores comparison

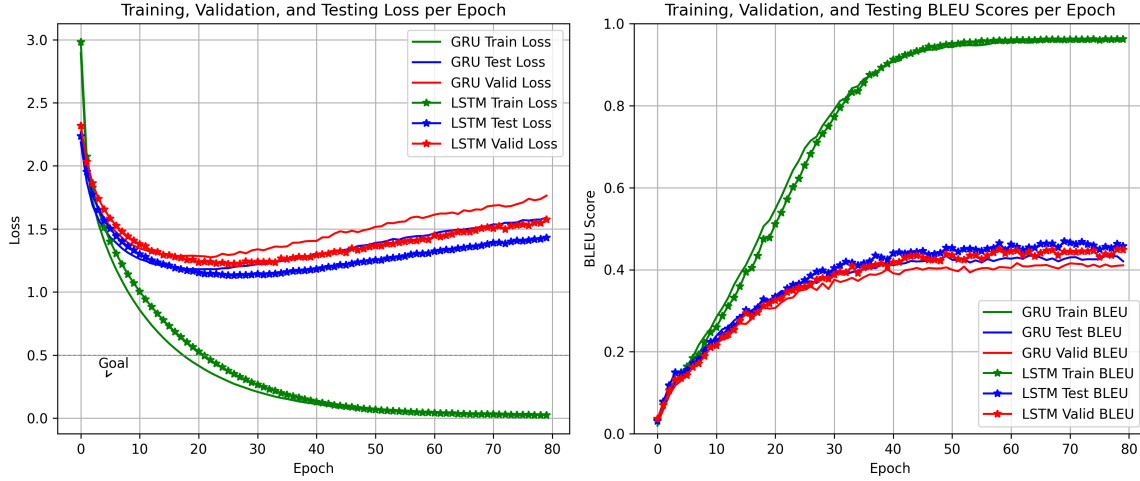


Figure 6: GRU and LSTM comparison

As we can see in our graph, the *loss function* of these two models is quite close to each other, even in terms of the *BLEU score*. This means that within the same dataset, the *encoder-decoder* translation achieves almost the same result (the difference can be ignored due to its small value). As we presented before, our proposed model GRU is similar to LSTM in its main concept but is simpler and less complicated. Therefore, as expected, the proposed model performs very similarly to LSTM in the same environment. Consequently, we would rather choose the simpler model over the more complex one.

4.2 Attention

We can also see that a potential issue with the encoder-decoder approach is that the neural network needs to compress all the necessary information of a source sentence into a *fixed-length* vector. This may make it difficult for the neural network to handle long sentences, especially those longer than the sentences in the training corpus. Additionally, this can lead to some errors in translation. The examples are as follows.

Source	Target	Translate
il est pauvre il est pauvre il est pauvre il est pauvre	he is poor he is poor he is poor he is poor	he is anxious to see her s safety <EOS>
il est pauvre	he is poor	he is poor but she s girl <EOS>
vous etes aveugle par l amour	you are blinded by love	you are trusted by every one of us problem <EOS>
j ecoute	i m listening	i m listening to bjork s latest song <EOS>

Table 1: fusion of 2 correct sentences where max lenght is 10.

Here, we will introduce an *Attention mechanism* to improve the performance of our model (we use Bahdanau-Attention to enhance our model). Now, let us look at the following *Attention graph* to view the attention distribution for each word.

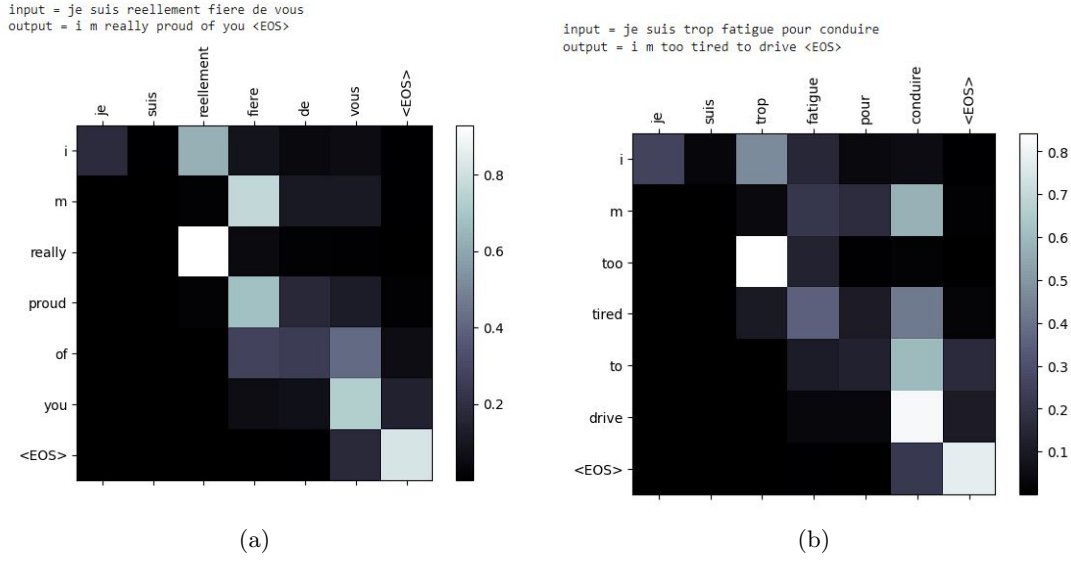


Figure 7: Attention graph

Here we can see the given *attention weight* of each words according to its context, and the following is the training result of our modified model:

GRU and LSTM (Attention) scores comparaision

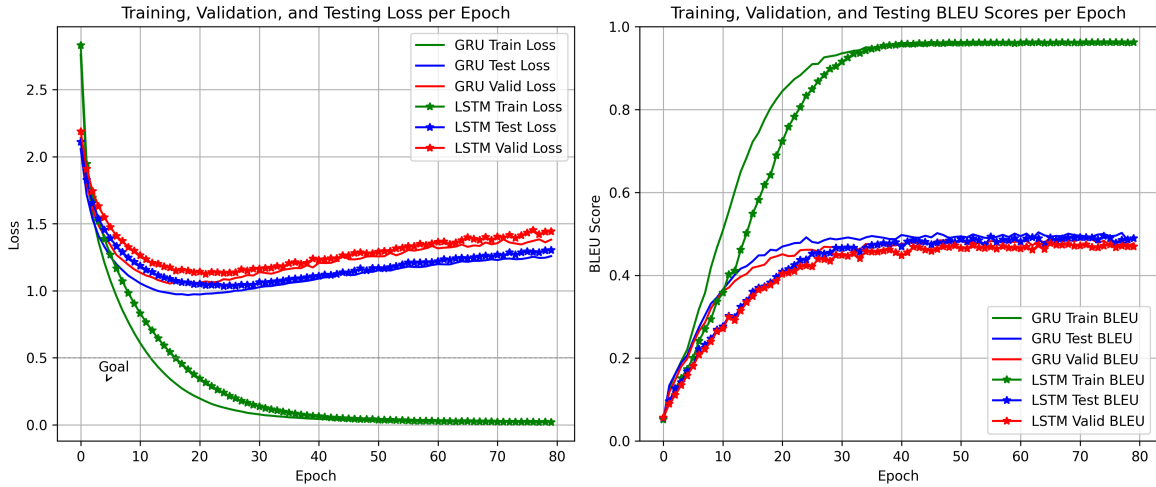


Figure 8: Enter Caption

As seen in this figure, both the *Loss function* and *BLEU score* of GRU and LSTM tend to improve, while the GRU performs better. Let us revisit the example from above:

Source	Target	Translate
il est pauvre il est pauvre il est pauvre il est pauvre	he is poor he is poor he is poor he is poor	he is poor as he is poor <EOS>
vous etes aveugle par l amour	you are blinded by love	you are blinded by love <EOS>
j ecoute	i m listening	i m listening and thirsty <EOS>
je viens du bresil	i am from brazil	i m from brazil <EOS>

Table 2: fusion of 2 correct sentences where max lenght is 10.

After that, we can see the improvement of our model (GRU). Although it is not perfect, it is still better than the previous one. With the *Attention mechanism*, the proposed model (GRU) seems to work better than LSTM. That is why the article introduces this method, which is simpler and less complex than LSTM. Even without *Attention*, the performance of both models is very close, with only a small margin of error.

4.3 STM

In this part we will compare the RNN encoder-decoder with STM (Statistical Machine Translation Model, proposed by IBM). First take a look at its *BLUE score*.

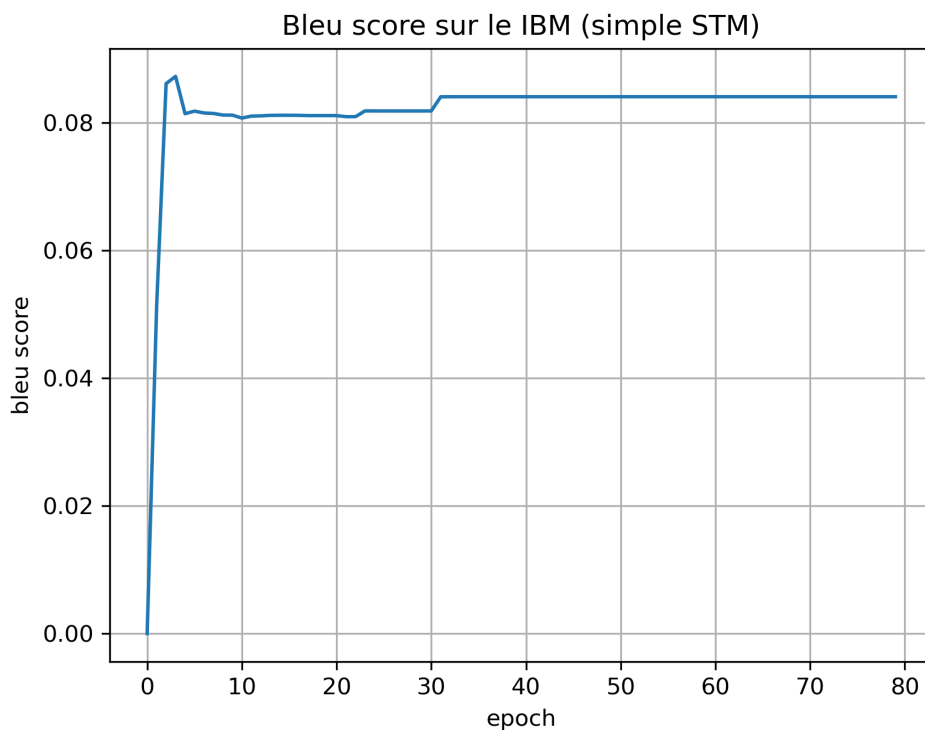


Figure 9: BLEU

Next, let us compare it with the GRU.

Source	IBM	GRU
nous sommes tous dingues	us we all jazz (with p=0.27)	we re all crazy <EOS>
elles ne sont pas encore chez elles	they don they t yet mcdonald they (with p=0.22)	they re not home yet <EOS>
je suis tres curieux	m i very curious (with p=0.42)	i am very curious <EOS>

Table 3: IBM VS GRU.

Under the same environment and the same number of iterations, we can see that the IBM model performs significantly worse and is less precise in terms of translation. Its *BLEU score* is not ideal. Here, we can compare the performance of the new model to SMT (in a simple case).

5 Conclusion

Compared to the complex structure of an LSTM, our proposed model GRU offers a simpler yet effective alternative within its RNN framework, achieving comparable results with fewer computational resources. This model introduces efficiencies that are particularly advantageous in machine translation tasks. Even when compared to SMT (IBM), our model demonstrates effective accuracy in sentence translation without the overhead typically associated with more complex architectures.

References

- [BCB14] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [CvMG⁺14] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, October 2014. Association for Computational Linguistics.
- [GS05] Alex Graves and Jürgen Schmidhuber. Framewise phoneme classification with bidirectional lstm and other neural network architectures. *Neural Networks*, 18(5):602–610, 2005. IJCNN 2005.
- [HS97] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9:1735–80, 12 1997.
- [PRWZ02] Kishore Papineni, Salim Roukos, Todd Ward, and Wei Jing Zhu. Bleu: a method for automatic evaluation of machine translation. 10 2002.

- [SVL14] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014.

A BLUE Score

Definition A.1. Given a string $y = y_1 y_2 \dots y_K$ and any integer $n \geq 1$, we define the set of its n -grams to be

$$G_n(y) = \{y_1 \dots y_n, y_2 \dots y_{n+1}, \dots, y_{K-n+1} \dots y_K\}.$$

Definition A.2. Given any two strings s, y , define the substring count $C(s, y)$ to be the number of appearances of s as a substring of y .

Example A.1. $C(ab, abcbab) = 2$.

Given one candidate sentence $\hat{y}$ and one reference sentence y , we have

$$p_n(\{\hat{y}\}; \{y\}) := \frac{\sum_{s \in G_n(\hat{y})} \min(C(s, \hat{y}), C(s, y))}{\sum_{s \in G_n(\hat{y})} C(s, \hat{y})}.$$

Now, fix a candidate corpus $\hat{S} := (\hat{y}^{(1)}, \dots, \hat{y}^{(M)})$, and reference candidate corpus $S = (S_1, \dots, S_M)$, where each $S_i := (y^{(i,1)}, \dots, y^{(i,N_i)})$.

Definition A.3. The (modified) **n**-gram precision function is

$$p_n(\hat{S}; S) := \frac{\sum_{i=1}^M \sum_{s \in G_n(\hat{y}^{(i)})} \min(C(s, \hat{y}^{(i)}), \max_{y \in S_i} C(s, y))}{\sum_{i=1}^M \sum_{s \in G_n(\hat{y}^{(i)})} C(s, \hat{y}^{(i)})}$$

Let c be the length of the candidate corpus, i.e. $c := \sum_{i=1}^M |\hat{y}^{(i)}|$, where $|y|$ is the length of y . Let r be the effective reference corpus length, i.e. $r := \sum_{i=1}^M |y^{(i,j)}|$, where $y^{(i,j)} = \arg \min_{y \in S_i} ||y| - |\hat{y}^{(i)}||$, the sentence from S_i whose length is as close to $|\hat{y}^{(i)}|$ as possible.

Definition A.4. The brevity penalty is

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-\frac{r}{c})} & \text{if } c \leq r \end{cases}$$

Definition A.5. Let $w := (w_1, w_2, \dots)$ be a weighting vector with $\sum_{i=1}^{\infty} w_i = 1$ and $w_i \in [0, 1]$ for all i in $\mathbb{N}^+$. With a choice of w , the BLEU score is

$$BLEU_w(\hat{S}; S) := BP(\hat{S}; S) \cdot \exp \left(\sum_{n=1}^{\infty} w_n \ln p_n(\hat{S}; S) \right)$$